

Class: GameCharacter						
Method	#	Test Description	Test Input	Expected Output	Actual Output	Status
GameCharacter()	1	Initializes fields	("Ruby",120.0,25.0,"Red like roses!")	Ruby,120.0,25.0,Red like roses	Ruby,120.0,25.0,Red like roses	PASS
charDeath()	1	Alive when HP>0	health = 85.0f	FALSE	FALSE	PASS
	2	Zero HP is dead	health = 0.0f	TRUE	TRUE	PASS
	3	Negative HP is dead	health = -10.0f	TRUE	TRUE	PASS
dealDmg()	1	Attacker deals damage	HP: 40.0f, Attack: 12.5f	HP: 27.5	HP: 27.5	PASS
	2	Zero attack does nothing	HP: 27.5f, Attack: 0.0f	HP: 27.5	HP: 27.5	PASS
	3	Self-hit reduces own HP	HP: 50.0f, Attack: 12.5f	HP: 37.5	HP: 37.5	PASS
takeDmg() [overriden method operate under the same logic, thus are collated into one method test]	1	Normal damage	health = 100.0f, damage = 15.0f	HP: 85.0	HP: 85.0	PASS
	2	Overkill damage	health = 30.0f, damage = 40.0f	HP: -10.0	HP: -10.0	PASS
	3	Negative damage	health = 50.0f, damage = -15.0f	HP: 65.0	HP: 65.0	PASS
nextTile() [overriden method operate under the same logic, thus are collated into one method test]	1	Next tile up	dir=0	(0,1)	(0,1)	PASS
	2	Next tile down	dir=1	(2,1)	(2,1)	PASS
	3	Next tile left	dir=2	(1,0)	(1,0)	PASS
	4	Next tile right	dir=3	(1,2)	(1,2)	PASS
move()	1	Move into passable 'h' tile to the right	dir=3	(1,2)	(1,2)	PASS
	2	Move down into destructible 'v' (blocked)	dir=1	(1,1)	(1,1)	PASS
	3	Overriden is true	dir=2 starting position: (1,0)	(1,0)	(1,0)	PASS

Class: PlayableChar						
Method	#	Test Description	Test Input	Expected Output	Actual Output	Status
PlayableChar()	1	Initializes playable char	("Hero", 50.0, 5.0)	name: Hero health: 50.0 attack: 5	name: Hero health: 50.0 attack: 5	PASS
addItem()	1	Add normal item	Rock	contains Rock: true	contains Rock: true	PASS
	2	Add consumable item	Potion	contains Potion: true	contains Potion: true	PASS
	3	Add null item	null	contains null: true	contains null: true	PASS
addAbility()	1	Add string	"Super speed"	contains "Super speed": true	contains "Super speed": true	PASS
	2	Add string concatenation	num = 1 "Ability " + num	contains "Ability 1": true	contains "Ability 1": true	PASS
	3	Add null String	null	contains null: true	contains null: true	PASS
hasAbility()	1	Existing ability	"Super speed"	TRUE	TRUE	PASS
	2	Ability cannot be found	" "	FALSE	FALSE	PASS
	3	Find string concatenation	"Super " + "speed"	TRUE	TRUE	PASS
incrementTurn()	1	Start of turn counter	0	1	1	PASS
	2	Turn counter at lower number	15	16	16	PASS
	3	Turn counter at high number	9000	9001	9001	PASS
takeDmg	1	Take damage	Damage = 1.5 Current health 3/3	Current health 1.5/3	Current health 1.5/3	PASS
	2	Take bat damage with "Bat Damage Reduction" ability	Bat damage = 1.5 Current health 3/3	Current health 2.5/3	Current health 2.5/3	PASS
	3	Character death	Damage = 1.5 Current health 1.5/3	Current health 0/3 Cause of death: "Bat"	Current health 0/3 Cause of death: "Bat"	PASS
	4	Revival mechanic	Damage = 1.5 Current health 1.5/3	Current health 3/3	Current health 3/3	PASS
heal()	1	Normal heal	20	30	30	PASS
	2	Over-max heal	100	50	50	PASS
	3	Negative heal	-10	50	50	PASS
useItem()	1	Use healing consumable	Noppo Bread, HP=2.5	HP=3 return TRUE	HP=3 return TRUE	PASS
	2	No item on hand	null	FALSE	FALSE	PASS
prevItem()	1	Normal cycle	current index = 3	index = 2	index = 2	PASS
	2	At first item (should cycle around)	current index = 0	index = 5	index = 5	PASS
	3	Scrolls into first item (should not jump to 0)	current index = 2	index = 1	index = 1	PASS
nextItem()	1	Normal cycle	current index = 0	index = 1	index = 1	PASS
	2	At final item (should cycle around)	current index = 5	index = 0	index = 0	PASS
	3	Scrolls into final item (should not jump to 0)	current index = 4	index = 5	index = 5	PASS
buyItem()	1	item is within gold owned	gold owned = 100 item price = 100	TRUE	TRUE	PASS
	2	gold owned is under item price	gold owned = 50 item price = 100	FALSE	FALSE	PASS
	3	null item	null	FALSE	FALSE	PASS
findCharTile()	1	character not found	map where no Y tile exists (assuming character symbol is Y)	(-1, -1)	(-1, -1)	PASS
	2	Find character position map	Y at (0,1)	(0,1)	(0,1)	PASS
collect()	1	Gold collect	<randomized> gold amount current gold 10	current gold 52	current gold 52	PASS
	2	Item collect	Noppo Bread item	current item Noppo Bread	current item Noppo Bread	PASS
dig()	1	replaces wall tile	current symbol 'v'	new symbol '.'	new symbol '.'	PASS
	2	replaces spike tile	current symbol 'x'	new symbol '.'	new symbol '.'	PASS
move()	1	Move char right	d current tile (1,1)	(1,2)	(1,2)	PASS
	2	Keep char in place	x current tile (1,1)	(1,1)	(1,1)	PASS
	3	Invalid movement	x current tile (1,1)	(1,1)	(1,1)	PASS
openTreasure()	1	Open treasure	toggles symbol to l/g	treasure l or g	l	PASS

Class: EnemyChar						
Method	#	Test Description	Test Input	Expected Output	Actual Output	Status
EnemyChar()	1	Construct enemy and verify fields	("Bat",8.0,3.0,10,1,1,2,2)	Bat,8.0,3.0,10,1,1,2,2	Bat,8.0,3.0,10,1,1,2,2	PASS
calcDistance	1	Whole number inputs	x1 = 1 y1 = 2 x2 = 9 y2 = 3	8.062	8.062	PASS
	2	Real number inputs	x1 = 1.5 y1 = 2.2 x2 = 8.6 y2 = 4.3	7.404	7.404	PASS
	3	Identical tiles	x1 = 1 y1 = 1 x2 = 1 y2 = 1	0	0	PASS
detectPlayer()	1	Detect player at adjacent tile	range=1 distance 1	TRUE	TRUE	PASS
	2	Player is outside range	range=1.5 distance 2	FALSE	FALSE	PASS
	3	Player is within diagonal range	range=1.5 distance 1.2	TRUE	TRUE	PASS
dropGold()	1	Drop gold tile on floor	goldDrop=10	g	g	PASS
	2	Drop gold at another coordinate	goldDrop=5	g	g	PASS
	3	Drop gold at third coordinate	goldDrop=15	g	g	PASS

	Class: Bat						
	Method	#	Test Description	Test Input	Expected Output	Actual Output	Status
	Bat()	1	Construct enemy and verify fields	(0.5, 5, 2, 1.0, false, 5, 5)	Bat,1.0, 5, 2, 1, false, 5, 5	Bat,1.0, 5, 2, 1, false, 5, 5	PASS
	move()	1	Player detected	Player health 3/3	Player health 2.5/3	Player health 2.5/3	PASS
		2	Random movement	n/a	<movement from 0-7>	4	PASS
		3	Blocked path (either by tile or another enemy)	current position (5, 5)	current position (5, 5)	current position (5, 5)	PASS

	Class: Siren						
	Method	#	Test Description	Test Input	Expected Output	Actual Output	Status
	Siren()	1	Construct enemy and verify fields	EnemyChar siren = new Siren(x, y);	"Siren", 1.0f, 10.0f, 750,1, 100, true, 5, 5, false	"Siren", 1.0f, 10.0f, 750,1, 100, true, 5, 5, false	PASS
	release()	1	Siren has yet to be released	n/a	TRUE	TRUE	PASS
		2	Siren is already released	n/a	TRUE	TRUE	PASS
	summonBat()	1	Free space	intended space (1,1)	new symbol 'b'	new symbol 'b'	PASS
		2	Space taken by another enemy	intended space (1,1)	new space != (1,1)	new space (3, 2)	PASS
		3	Space taken by player	intended space (1,1)	new space != (1,1)	new space (7, 12)	PASS
	move()	1	Player detected	Player at (1,1) Siren at (1, 5)	Player at (1,1) Siren at (1, 4)	Player at (1,1) Siren at (1, 4)	PASS
		2	Player adjacent	Player health 3/3	Player health 0/3	Player health 0/3	PASS
		3	Blocked path (either by tile or another enemy)	current position (5, 5)	current position (5, 5)	current position (5, 5)	PASS

	Class: NPChar						
	Method	#	Test Description	Test Input	Expected Output	Actual Output	Status
	NPChar()	1	Construct NPC with reward item	("Shopkeeper","Thank you!", reward)	Shopkeeper,Thank you!,Gift,5,false	Shopkeeper,Thank you!,Gift,5,false	PASS
	incrementTimesSaved()	1	Start of turn counter	0	1	1	PASS
		2	Turn counter at lower number	15	16	16	PASS
		3	Turn counter at high number	9000	9001	9001	PASS

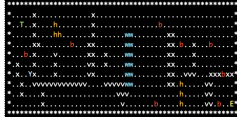
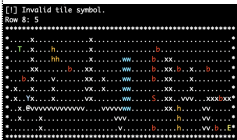
	Class: Item						
	Method	#	Test Description	Test Input	Expected Output	Actual Output	Status
	Item()	1	Construct item with default price	("Noppo Bread", null)	Noppo Bread, 0, null, true	Noppo Bread, 0, null, true	PASS
		2	Construct item with price	("Noppo Bread, 25", null)	Noppo Bread, 25, null, true	Noppo Bread, 25, null, true	PASS
	isUnlocked()	1	The item is not connected to any NPC	null	TRUE	TRUE	
		2	Connected NPC has yet to be saved	NPC saved = false	FALSE	FALSE	
		3	Connected NPC is saved	NPC saved = true	TRUE	TRUE	

	Class: ConsumableItem						
	Method	#	Test Description	Test Input	Expected Output	Actual Output	Status
	ConsumableItem()	1	Construct consumable with healAmount	("Noppo Bread", 15.0, null)	Noppo Bread, 0, 15.0, null, true	Noppo Bread, 0, 15.0, null, true	PASS
		2	Construct consumable with price and heal	("Noppo Bread",15,15.0, null)	Noppo Bread, 15, 15.0, null, true	Noppo Bread, 15, 15.0, null, true	PASS
	use()	1	Consumable heals player	NoppoBread.use()	TRUE	TRUE	PASS
		2	Consumable caps at max health	NoppoBread.use() when at 15/20	TRUE	TRUE	PASS

	Class: Passiveltem						
	Method	#	Test Description	Test Input	Expected Output	Actual Output	Status
	Passiveltem()	1	Constructor	("Air Shoes", 500, "Water and Heat Immunity", null)	Air Shoes, 500, "Water and Heat Immunity", null, true	Air Shoes, 500, "Water and Heat Immunity", null, true	PASS
	activate()	1	Adds ability to player	new PlayableChar("Yohane", 3.0f, 1.0f) to add: "Water and Heat Immunity"	Yohane.hasAbility("Water and Heat Immunity") = true	Yohane.hasAbility("Water and Heat Immunity") = true	PASS
		2	Player is null	null to add: "Water and Heat Immunity"	Yohane.hasAbility("Water and Heat Immunity") = false	Yohane.hasAbility("Water and Heat Immunity") = false	PASS

	Class: Upgradeltem						
	Method	#	Test Description	Test Input	Expected Output	Actual Output	Status
	Upgradeltem()	1	Constructor	("Magic Berry", 50, "Max Health", 2.0f, null)	Magic Berry, 50, Max Health, 2.0, null, true	Magic Berry, 50, Max Health, 2.0, null, true	PASS
	applyUpgrade()	1	Adds Max Health to player	new PlayableChar("Yohane", 3.0f, 1.0f) to add: "Max Health"	Yohane previous health 3 Yohane upgraded health 5	Yohane previous health 3 Yohane upgraded health 5	PASS
		2	Adds Attack to player	new PlayableChar("Yohane", 3.0f, 1.0f) to add: "Attack"	Yohane previous attack 3 Yohane upgraded attack 4	Yohane previous attack 3 Yohane upgraded attack 4	PASS
		3	Player is null	null to upgrade: "Max Health"	Yohane.hasAbility("Water and Heat Immunity") = false	Yohane.hasAbility("Water and Heat Immunity") = false	PASS

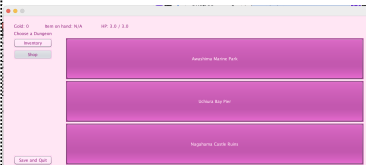
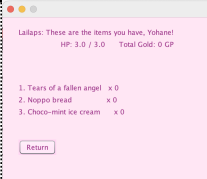
Class: Dungeon						
Method	#	Test Description	Test Input	Expected Output	Actual Output	Status
Dungeon()	1	Basic constructor	//supplementary instantiation Floor[] floors = new Floor[3]; floors[0] = new Floor(1); floors[1] = new Floor(2); floors[2] = new Floor(3); Item noppoBread = new Item ("Noppo Bread", 1000000); NPChar rando = new NPChar("KARA", null, noppoBread); //dungeon instatiation Dungeon dungeon = new Dungeon("TEST DUNGEON", 1, 3, floors, randu);	NAME: "TEST DUNGEON" DUNGEON NUM: 1 NUM FLOORS: 3 FLOORS: 3 MEMBER: "KARA" (this.member. getName())	NAME: "TEST DUNGEON" DUNGEON NUM: 1 NUM FLOORS: 3 FLOORS: 3 MEMBER: "KARA" (this.member. getName())	PASS
isCompleted()	1	Only floor 1 and 2 are completed	N/A	FALSE	FALSE	PASS
	2	Incomplete final floor	N/A	FALSE	FALSE	PASS
	3	Final floor completed	N/A	TRUE	TRUE	PASS
incrementCurFloor()	1	Increment from Floor 1	N/A	2	2	PASS
	2	Increment from Floor 2	N/A	3	3	PASS
	3	Edge case; increment from Floor 3 (should not increment)	N/A	3	3	PASS
gameOver()	1	0 health	Yohane health = 0.0f	TRUE	TRUE	PASS
	2	Full health	Yohane health = 3.0f	FALSE	FALSE	PASS
	3	Very small real number	Yohane health = 0.01f	TRUE	FALSE	PASS

Class: Floor						
Method	#	Test Description	Test Input	Expected Output	Actual Output	Status
Floor()	1	Basic constructor	Floor floor = new Floor(1)	MAP: <see generateFloor(> ENEMIES: 8 (enemies().size()) ROWLEN: 12 COLLEN: 65 FLOORNUM: 1	MAP: <see generateFloor(> ENEMIES: 8 (enemies().size()) ROWLEN: 12 COLLEN: 65 FLOORNUM: 1	PASS
generateFloor()	1	All valid tiles in map text file	map1.txt			PASS
	2	Invalid tile flagging	test.txt	[!] Invalid tile symbol. Row 8: 5		PASS
	3	File does not exist in directory (error testing)	invalid.txt	[!] File not found. Map could not be generated.	[!] File not found. Map could not be generated.	PASS
generateEnemues()	1	Standard amount of enemies on map	map1.txt	floor.getEnemies().size() == 8	floor.getEnemies().size() == 8	PASS
	2	Greater amount of enemies on map	test.txt	floor.getEnemies().size() == 10	floor.getEnemies().size() == 10	PASS
	3	No enemies on map	test2.txt	floor.getEnemies().size() == 0	floor.getEnemies().size() == 0	PASS
validateMove()	1	Impassable tile	map1.txt, tile at (0, 0); BORDER TILE	FALSE	FALSE	PASS
	2	Out of bounds tile	Tile OOB = new Tile(100, 100, '');	FALSE	FALSE	PASS
	3	Passable tile	map1.txt, tile at (1, 1); PASSABLE TILE	TRUE	TRUE	PASS
destroyTile()	1	Destroying spike tile	map1.txt, tile at (1, 6); SPIKE TILE	SYMBOL BEFORE: 'x' SYMBOL AFTER: ''	SYMBOL BEFORE: 'x' SYMBOL AFTER: ''	PASS
	2	Destroying wall tile	map1.txt, tile at (10, 27); WALL TILE	SYMBOL BEFORE: 'v' SYMBOL AFTER: ''	SYMBOL BEFORE: 'v' SYMBOL AFTER: ''	PASS
completeFloor()	1	Incomplete floor	Yohane at (5, 25) Exit at (10, 53)	FALSE	FALSE	PASS
	2	Complete floor	Yohane at (10, 53) Exit at (10, 53)	TRUE	TRUE	PASS
	3	Yohane is near Exit Tile	Yohane at (9, 53) Exit at (10, 53)	FALSE	FALSE	PASS
findEnemy()	1	Bat enemy found at tile	map1.txt, tile at (5, 4); BAT	Bat enemy	Bat enemy	PASS
	2	Enemy not found at tile	map1.txt, tile at (3, 4); PASSABLE TILE	null	null	PASS
	3	Siren enemy found at tile	test.txt, tile at (7, 35); SIREN	Siren enemy	Siren enemy	PASS
tileTaken()	1	Free tile	(5, 5), passable tile	FALSE	FALSE	PASS
	2	Tile is taken by another bat	(5, 5), bat tile	TRUE	TRUE	PASS
	3	Tile is taken by Siren	(5, 5), siren tile	TRUE	TRUE	PASS

Class: Tile						
Method	#	Test Description	Test Input	Expected Output	Actual Output	Status
Tile()	1	Basic constructor	Tile water = new Tile(0, 0, 'w');	X: 0 Y: 0 SYMBOL: 'w' PASSABLE: false DESTRUCTIBLE: false DAMAGE: 0	X: 0 Y: 0 SYMBOL: 'w' PASSABLE: false DESTRUCTIBLE: false DAMAGE: 0	PASS
assignColor()	1	water tiles	this.Symbol == 'w'	BLUE: w	BLUE: w	PASS
	2	heat tiles	this.Symbol == 'h'	ORANGE: h	ORANGE: h	PASS
	3	treasure tiles	this.Symbol == 'T'	GREEN: T	GREEN: T	PASS
	4	exit tiles	this.Symbol == 'E'	YELLOW: E	YELLOW: E	PASS
	5	gold tiles	this.Symbol == 'g'	YELLOW: g	YELLOW: g	PASS
	6	item tiles	this.Symbol == 'I'	TURQUOISE: I	TURQUOISE: I	PASS
	7	non-colored tiles	this.Symbol == '.'	WHITE N/A: .	WHITE N/A: .	PASS
assignProperties()	1	water tiles	this.Symbol == 'w'	passable = false destructible = false damage = 0.0	passable = false destructible = false damage = 0.0	PASS
	2	heat tiles	this.Symbol == 'h'	passable = true destructible = false damage = 1.0	passable = true destructible = false damage = 1.0	PASS
	3	treasure tiles	this.Symbol == 'T'	passable = false destructible = true damage = 0.0	passable = false destructible = true damage = 0.0	PASS
	4	gold tiles	this.Symbol == 'g'	passable = true destructible = true damage = 0.0	passable = true destructible = true damage = 0.0	PASS
	5	item tiles	this.Symbol == 'I'	passable = true destructible = false damage = 0.0	passable = true destructible = false damage = 0.0	PASS
	6	exit tiles	this.Symbol == 'E'	passable = true destructible = false damage = 0.0	passable = true destructible = false damage = 0.0	PASS
	7	passable tiles	this.Symbol == '.'	passable = true destructible = false damage = 0.0	passable = true destructible = false damage = 0.0	PASS
	8	border tiles	this.Symbol == '*'	passable = false destructible = false damage = 0.0	passable = false destructible = false damage = 0.0	PASS
	9	wall tiles	this.Symbol == 'v'	passable = false destructible = true damage = 0.0	passable = false destructible = true damage = 0.0	PASS
	10	spike tiles	this.Symbol == 'x'	passable = false destructible = true damage = 0.5	passable = false destructible = true damage = 0.5	PASS

Class: DestructibleTile						
Method	#	Test Description	Test Input	Expected Output	Actual Output	Status
DestructibleTile()	1	Basic constructor	DestructibleTile dT1 = new DestructibleTile(treasure);	X: 0 Y: 0 SYMBOL: 'T' PASSABLE: false DESTRUCTIBLE: true DAMAGE: 0 TREASURE: true GOLDDROP: 0-100 ITEMDROP: Noppo Bread	X: 0 Y: 0 SYMBOL: 'T' PASSABLE: false DESTRUCTIBLE: true DAMAGE: 0 TREASURE: true GOLDDROP: 26 ITEMDROP: Noppo Bread	PASS
	2	Copy constructor from parent tile	DestructibleTile dT3 = new DestructibleTile(0, 0, 'g', bat.getGoldDrop(), null, false);	X: 0 Y: 0 SYMBOL: 'g' PASSABLE: true DESTRUCTIBLE: true DAMAGE: 0 TREASURE: false GOLDDROP: 5 ITEMDROP: null	X: 0 Y: 0 SYMBOL: 'g' PASSABLE: true DESTRUCTIBLE: true DAMAGE: 0 TREASURE: false GOLDDROP: 5 ITEMDROP: null	PASS
dropTreasure()	1	Gold drop	Randomizer == 0	SYMBOL BEFORE: 'T' SYMBOL AFTER: 'g'	SYMBOL BEFORE: 'T' SYMBOL AFTER: 'g'	PASS
	2	Item drop	Randomizer == 1	SYMBOL BEFORE: 'T' SYMBOL AFTER: 'I'	SYMBOL BEFORE: 'T' SYMBOL AFTER: 'I'	PASS
assignProperties()	1	Treasure tile	DestructibleTile dT1 = new DestructibleTile(treasure);	this.treasure = true this.goldDrop = 10-100 this.itemDrop = "Noppo Bread"	this.treasure = true this.goldDrop = 26 this.itemDrop = "Noppo Bread"	PASS
	2	Heat tile	DestructibleTile dT2 = new DestructibleTile(spike);	this.treasure = false this.goldDrop = 0 this.itemDrop = null	this.treasure = false this.goldDrop = 0 this.itemDrop = null	PASS
	3	Gold tile	DestructibleTile dT3 = new DestructibleTile(0, 0, 'g', bat.getGoldDrop(), null, false);	this.treasure = false this.goldDrop = 5 this.itemDrop = null	this.treasure = false this.goldDrop = 5 this.itemDrop = null	PASS

	Class: Initialize						
	Method	#	Test Description	Test Input	Expected Output	Actual Output	Status
Initialize		1	Constructor with no parameters	DestructibleTile dT1 = new DestructibleTile(treasure);	X: 0 Y: 0 SYMBOL: 'T' PASSABLE: false DESTRUCTIBLE: true DAMAGE: 0 TREASURE: true GOLDDROP: 0-100 ITEMDROP: Noppo Bread	X: 0 Y: 0 SYMBOL: 'T' PASSABLE: false DESTRUCTIBLE: true DAMAGE: 0 TREASURE: true GOLDDROP: 26 ITEMDROP: Noppo Bread	PASS
		2	Constructor with given NPCs and items	DestructibleTile dT3 = new DestructibleTile(0, 0, 'g', bat, getGoldDrop(), null, false);	X: 0 Y: 0 SYMBOL: 'g' PASSABLE: true DESTRUCTIBLE: true DAMAGE: 0 TREASURE: false GOLDDROP: 5 ITEMDROP: null	X: 0 Y: 0 SYMBOL: 'g' PASSABLE: true DESTRUCTIBLE: true DAMAGE: 0 TREASURE: false GOLDDROP: 5 ITEMDROP: null	PASS
initializeNPCs()		1	Complete, unsaved roster	n/a	- Kanan Matsuura (Saved: false) - Hanamaru Kunikida (Saved: false) - Ruby Kurosawa (Saved: false) - Dia Kurosawa (Saved: false) - Chika Takami (Saved: false) - You Watanabe (Saved: false) - Riko Sakurauchi (Saved: false) - Mari Ohara (Saved: false)	- Kanan Matsuura (Saved: false) - Hanamaru Kunikida (Saved: false) - Ruby Kurosawa (Saved: false) - Dia Kurosawa (Saved: false) - Chika Takami (Saved: false) - You Watanabe (Saved: false) - Riko Sakurauchi (Saved: false) - Mari Ohara (Saved: false)	PASS
initializeItems()		1	Complete item list with correct prices	n/a	- Tears of a fallen angel Price: 30 Available: true - Noppo Bread Price: 100 Available: true - Shovel Upgrade Price: 300 Available: true - Bat Tamer Price: 400 Available: true - Air Shoes Price: 500 Available: true - Stewshine Price: 1000 Available: true - Mikan Mochi Price: 1000 Available: true - Kurosawa Matcha Price: 1000 Available: true - Choco-Mint Ice Cream Price: 2000 Available: true	- Tears of a fallen angel Price: 30 Available: true - Noppo Bread Price: 100 Available: true - Shovel Upgrade Price: 300 Available: true - Bat Tamer Price: 400 Available: true - Air Shoes Price: 500 Available: true - Stewshine Price: 1000 Available: true - Mikan Mochi Price: 1000 Available: true - Kurosawa Matcha Price: 1000 Available: true - Choco-Mint Ice Cream Price: 2000 Available: true	PASS
initializeDungeons()		1	Dungeons with increasing floors. No details should be the same	n/a	Dungeon1.name != Dungeon2.name Dungeon2.name != Dungeon3.name Dungeon1.name != Dungeon3.name (same for connected NPC) Dungeon1 -> 1 floor Dungeon2 -> 2-3 floor Dungeon3 -> 3-4 floor	- Nagahama Castle Ruins Floors: 1 Rescue Target: Ruby Kurosawa - Shougetsu Confectionary Floors: 2 Rescue Target: Hanamaru Kunikida - Awashima Marine Park Floors: 3 Rescue Target: Mari Ohara	PASS
		2	Boss dungeon generation	n/a	- Siren of Numazu Floors: 1 Rescue Target: None (Boss Dungeon)	- Siren of Numazu Floors: 1 Rescue Target: None (Boss Dungeon)	PASS
assignFloors()		1	Floor generation. No map files should be the same.	Force the method to run 8 times	No file name should be the same	map1.file = "map4.txt" map2.file = "map2.txt" map3.file = "map8.txt" map4.file = "map1.txt" map5.file = "map3.txt" map6.file = "map7.txt" map7.file = "map5.txt" map8.file = "map6.txt"	PASS
assignNPC()		1	Unrecognized dungeon name	"Fake dungeon"	null	null	PASS
		2	Recognized dungeon name	"Shougetsu Confectionary"	"Hanamaru Kunikida"	"Hanamaru Kunikida"	PASS
		3	Siren's dungeon	"Siren of Numazu"	null	null	PASS

Package: View						
Class	#	Test Description	Test Input	Expected Output	Actual Output	Status
MainMenuPanel	1	Display main menu	n/a	Displayed window		PASS
GameMenuPanel	1	Display GameMenuPanel	n/a	Dislplayed window		PASS
InventoryPanel	1	Display inventory	n/a	Displayed window		PASS
GamePanel	1	Display Game	n/a	Dislplayed window		PASS
ShopPanel	1	Display shop	n/a	Dislplayed window		PASS
StatusPanel	1	Display status	n/a	Dislplayed window		PASS